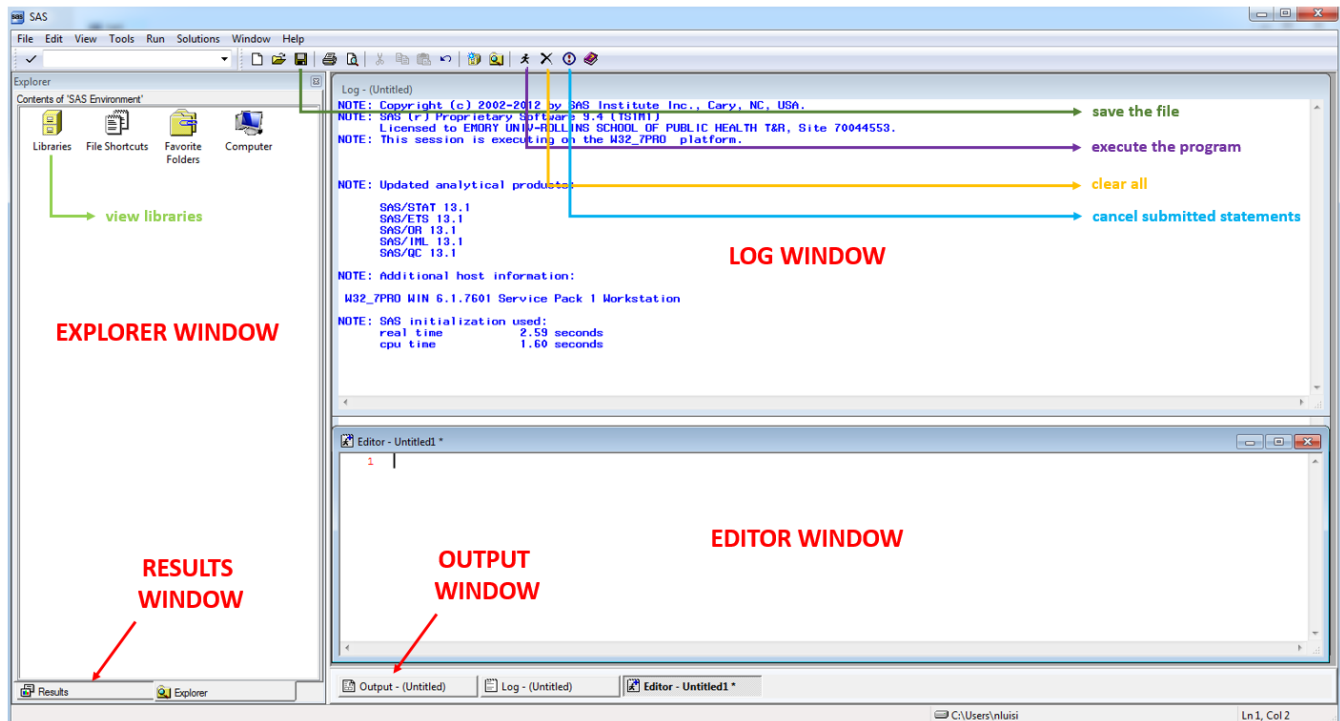


Module 1

Topics: Introduction to SAS, SAS dataset basics, SAS explorer applications, SAS customizations, best practices

1.1 - Introduction to SAS



Editor Window. The Editor Window is where you type in your SAS programs. You can also open previously saved SAS programs, as well as save new SAS programs. After you have typed in your SAS program, and are satisfied that it meets all of the syntax requirements, you can "run" (or "execute") your program by clicking on the "running man" icon . For Windows operating systems, the default editor is called the Enhanced Editor, because it uses differently colored text to warn you if your program contains a syntax error.

Log Window. The Log Window displays messages about your SAS session and any programs that you submit. You should *always* plan to check this window after running a program. Even though your program may appear to have run correctly, critical errors may still have occurred when reading or manipulating the data. SAS uses the following color-coded system to assist you in reading the log:

- the DATA and PROC steps that appear in your program are printed in **black**
- notes that SAS wants to report to you are printed in **blue**
- warnings that SAS wants to draw to your attention are printed in **green**
- and errors that cause SAS to abort running your program are printed in **red**

Output Window. The Output Window is where the printable results from your program appear. It is positioned behind the Log and Editor Windows until there is output to display, when it automatically opens or moves to the front of your display. Examples of output that your programs might create include data listings, table summaries, charts, and character-based plots and graphs. If you review the Output Window after running a program and some output that you expected is missing, then double-check the Log Window to see if you had any programming errors that prevented SAS from executing your commands. Note, though, that not all SAS programs create output in the Output Window. If you create HTML output, for example, it can be viewed in the internal SAS browser called the Results Viewer Window. And, if you create a graph, it will appear in a separate Graph Window.

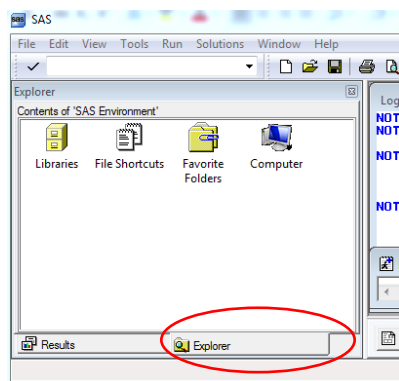
Note that if your program is creating output in the Results Viewer Window, by default, you will want to turn this feature off. Make this change under preferences (found in the tools menu, under options). In the Preferences Window, select the Reports Tab. You will want to uncheck Create HTML and, instead, select Create Listing. This is important, because when SAS produces HTML it takes control of how the output looks out of your hands.

Explorer Window. The Explorer Window allows you to easily view and manage your SAS files, which are stored in SAS data libraries. We'll learn more about data libraries later. For now, it suffices to know that a library name is just a nickname for the actual location — that is, a folder on your computer — of your SAS files. The Explorer Window can be used to create new SAS libraries and files, to open SAS files, and to move, copy and delete SAS files.

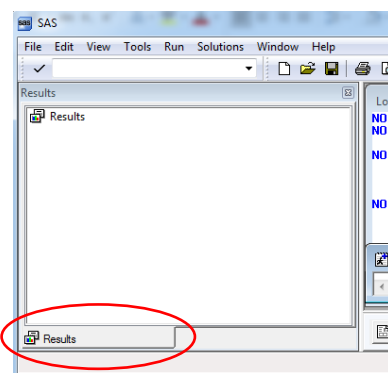
Results Window. (Note that the Results Window is not the same as the Results Viewer Window described previously.) The Results Window serves much like a table of contents for your Output Window. That is, it itemizes each section of your Output Window in outline form so you can easily jump from one piece of output to another. The Results Window is empty until you submit a SAS program that creates output. Then, it moves to the front of your display.

Tips:

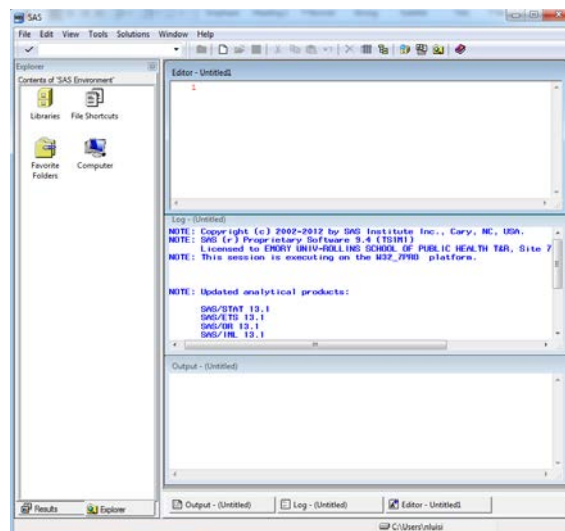
- If you accidentally close your Explorer window, you can restart SAS OR open a new explorer window (View → Explorer) and then dock it (Window → Docked). The “Contents Only” option looks most like the original view when you open a new SAS session.



OH NO!

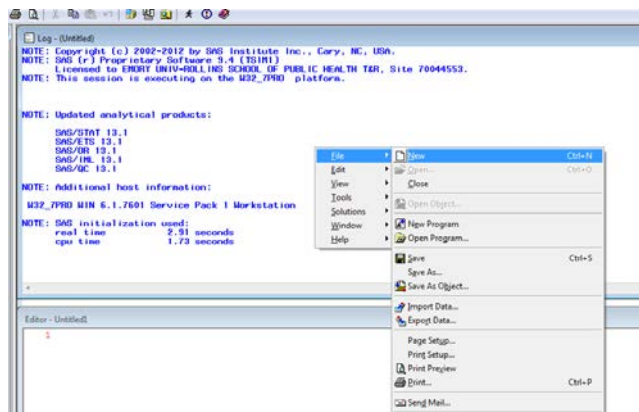


- You can also customize the view with options in the Window menu.



For example, “Tile Horizontally” will stack the Editor, Log, and Output windows.

- If your log is cluttered you can clear it by right clicking in the log window and selecting File → New.



1.2 – Dataset Basics

The purpose of any dataset is to store information. Database structure has been standardized and is often described as a rectangular spreadsheet with rows and columns. The rows contain the observations (subjects) and the columns contain the variables (characteristics of the subjects). Shown below is an example of a small dataset.

ID	NAME	WEIGHT
1001	John	125
1002	Paul	.
1003		178
1004	George	142
1005	Ringo	120

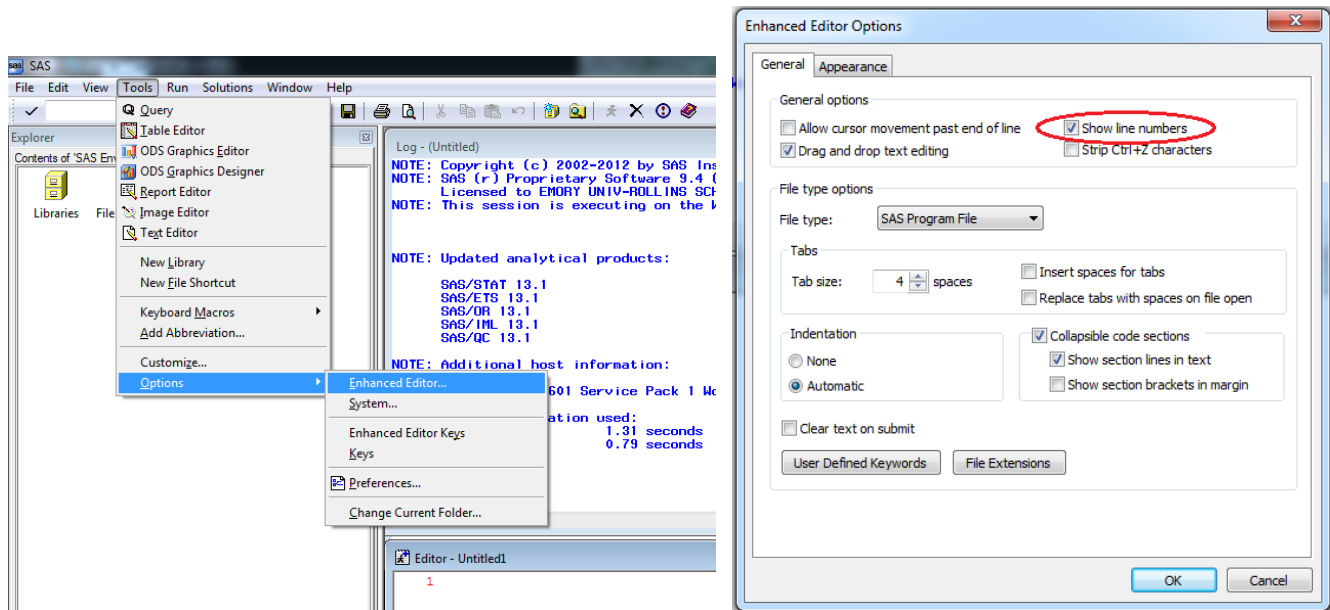
The value “John” is stored in the variable called NAME for the first observation in the dataset. The NAME variable in this dataset is storing character data which is one of the two types of data stored in SAS datasets. The variable ID in this dataset is storing the other type of data, namely numeric. Observation number two has missing information for weight and observation number three has missing data on name. In a SAS dataset, missing data are stored as very small negative numbers and displayed as a blank for character data and as a period for numeric data.

1.3 - SAS Customization

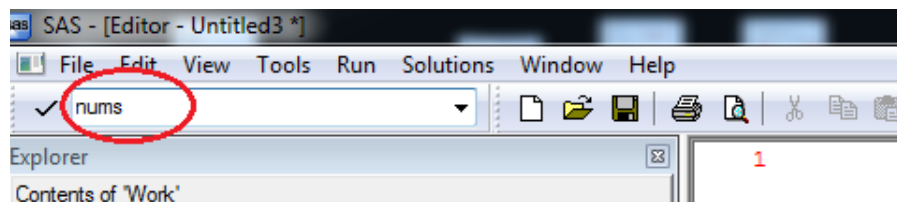
SAS sessions can be customized (fonts, colors, shortcuts, and other preferences). This is handy if you are using your own copy of SAS (installed on your computer), but might not be worth doing if you are using SAS in a virtual environment like Citrix if everything goes back to the default settings each time you log out of the system. These are some of my favorite customizations.

Line Numbers

Line numbers are helpful when you want to direct someone to look at a specific part of your program. To add line numbers, first click somewhere in the program editor window, then go to Tools → Options → Enhanced Editor. In the General tab, check the box to ‘Show line numbers’.



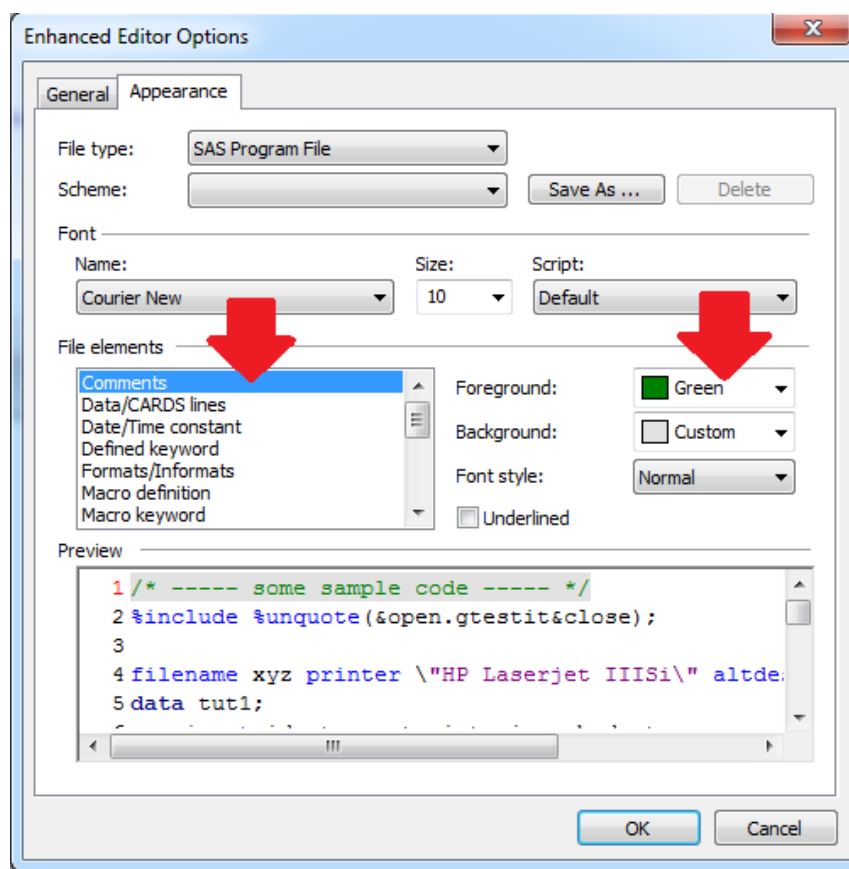
Another shortcut for this (one that might be helpful if you are using Citrix and can't save these types of settings) is to use the command box in the toolbar at the top left of the screen. When you are in the editor, type “nums” in the command box and hit enter. Line numbers should appear in the editor window. Type “nums” and hit enter again to turn line numbers off.



Comments

You should always use comments in your SAS code when possible – I often use comments to explain what each section of code is doing or provide additional details about dataset updates/cleaning. Comments are very helpful to other people that inherit your code and can help to keep you organized or refresh your memory later on.

By default, SAS makes comment text green, but I like my comments to stand out a bit more. In this example I am making my comments green with a grey background. Again, you will first click somewhere in the program editor window, then go to Tools → Options → Enhanced Editor. This time choose the Appearance tab. Choose the element you want to edit (in this case Comments) and select a foreground and background color.



SAS will accept either of these as comments:

```
* This is a comment. ;
```

```
/* This is also a comment. */
```

As a shortcut to comment a section of code, you can select the text you want to comment and hit the CTRL key and / key simultaneously. This will comment each line selected. To undo this, you can select the text again and hit the CTRL key and SHIFT key and / key simultaneously.

```
data something;
    set somethingelse;
    if thisthing=0 then thatthing=1;
run;

/*data something;*/
/*    set somethingelse;*/
/*    if thisthing=0 then thatthing=1;*/
/*run;*/
```

1.4 – Best Practices

Formatting your SAS programs

SAS is not terribly picky about the layout of your DATA or PROC steps. You can write an entire program in one line, or start each statement on a new line (as long as you don't try to split words in two). The semicolon tells SAS when a statement is complete (and RUN tells SAS when to execute it).

Examples 1, 2, and 3 below contain the same DATA step in SAS. Example 1 is valid and would produce the same result as Example 2, but Example 2 is much easier to read and follow because of the indenting. Examples 2 and 3 are meant to demonstrate that SAS statements are not case sensitive.

Note that you can't run the code below in SAS, these data steps are only examples.

Example 1:

```
data something; set otherthing; if likesas = 1 then yay = 1; else yay = 0; run;
```

Example 2:

```
data something;
    set otherthing;
        if likesas = 1 then yay = 1;
        else yay = 0;
run;
```

Example 3:

```
DATA SOMETHING;
    SET OTHERTHING;
        IF LIKESAS = 1 THEN YAY = 1;
        ELSE YAY = 0;
RUN;
```

Running your SAS programs

Some of the most challenging issues for new SAS users include improper dataset overwriting and inefficient data step combinations. New users tend to use more data steps than necessary to accomplish a task, which can be okay as long as the user is careful and the datasets are properly named.

Note that you can't run the code below in SAS, these data steps are only examples.

Example 1:

Q: What is TEMP 1 going to look like?

```
DATA TEMP1;
    SET NICOLE.ORIGINAL;
    IF VARA=20 THEN VARA=123;
RUN;

DATA TEMP1;
    SET NICOLE.ORIGINAL;
    IF VARB LT 4 OR VARB GT 6 THEN VARB=. ;
RUN;

DATA TEMP1;
    SET NICOLE.ORIGINAL;
    IF VARC=16 THEN DO;
        VARD=0;
        VARD = VARD+1;
    END;
RUN;

DATA TEMP1;
    SET NICOLE.ORIGINAL;
    IF VARE GT 16 THEN VARE=.;

    IF VARF GT 7 THEN VARF=.;

    IF VARG = 'less' OR VARG = 'LESS' OR VARG = 'Less' THEN VARH = 0;
    IF VARG = 'some' OR VARG = 'SOME' THEN VARH = 1;
    IF VARG = 'more' OR VARG = 'MORE' THEN VARH = 2;
RUN;
```

A: Because each step above is creating WORK.TEMP1 from NICOLE.ORIGINAL, once all steps are run, TEMP1 will have ONLY the changes made in the final data step (VARE, VARF, VARG, and VARH). No changes made to VARA through VARD will be included; it is as if they never happened. This is because with each new data step, we used the same NICOLE.ORIGINAL dataset and did not retain any of our work from the previous data steps. If we had any errors or problems along the way, it would be difficult to find them and we would have to start back at the beginning with the original file every time since we kept overwriting TEMP1 in this series of steps.

Example 2:

Q: Why is this a terrible idea?

```
DATA TEMP1;
    SET NICOLE.ORIGINAL;
    IF VARA=20 THEN VARA=123;
```

AEPI537D

```

RUN;

DATA TEMP1;
    SET TEMP1;
    IF VARB LT 4 OR VARB GT 6 THEN VARB=. ;
RUN;

DATA TEMP1;
    SET TEMP1;
    IF VARC=16 THEN DO;
        VARD=0;
        VARD = VARD+1;
    END;
RUN;

DATA TEMP1;
    SET TEMP1;
    IF VARE GT 16 THEN VARE=.;

    IF VARF GT 7 THEN VARF=.;

    IF VARG = 'less' OR VARG = 'LESS' OR VARG = 'Less' THEN VARH = 0;
    IF VARG = 'some' OR VARG = 'SOME' THEN VARH = 1;
    IF VARG = 'more' OR VARG = 'MORE' THEN VARH = 2;
RUN;

```

A: Although the first step above is creating WORK.TEMP1 from NICOLE.ORIGINAL, all other steps are replacing TEMP1 with another file of the same name. If we had any errors or problems along the way, it would be difficult to find them and we would have to start back at the beginning with the original file every time since we kept overwriting TEMP1 in this series of steps.

Example 3:

Q: Can we keep the same number of datasets as Example 1 but do this a slightly better way?

```

DATA TEMP1a;
    SET NICOLE.ORIGINAL;
    IF VARA=20 THEN VARA=123;
RUN;

DATA TEMP1b;
    SET TEMP1a;
    IF VARB LT 4 OR VARB GT 6 THEN VARB=. ;
RUN;

DATA TEMP1c;
    SET TEMP1b;
    IF VARC=16 THEN DO;
        VARD=0;
        VARD = VARD+1;
    END;
RUN;

DATA TEMP1;
    SET TEMP1c;
    IF VARE GT 16 THEN VARE=.;

    IF VARF GT 7 THEN VARF=.;

    IF VARG = 'less' OR VARG = 'LESS' OR VARG = 'Less' THEN VARH = 0;
    IF VARG = 'some' OR VARG = 'SOME' THEN VARH = 1;
    IF VARG = 'more' OR VARG = 'MORE' THEN VARH = 2;
RUN;

```


A: Yes! In Example 3, we used the same number of data steps as Examples 1 and 2, but we had UNIQUE names for each new dataset we created, and we always set from the dataset in the previous step. This way we retained the changes we made to each dataset and continued to build on them with each new data step. In Example 3 we still end up with a dataset called TEMP1, but this one has all the changes we made in each step. And if we had any errors or problems along the way, we could find them and fix them without starting from the very beginning with the original file.

Example 4:

Q: So what's the best way to do this?

```
DATA TEMP1;
    SET NICOLE.ORIGINAL;
    IF VARA=20 THEN VARA=123;
    IF VARB LT 4 OR VARB GT 6 THEN VARB=. ;
    IF VARC=16 THEN DO;
        VARD=0;
        VARD = VARD+1;
    END;
    IF VARE GT 16 THEN VARE=.;
    IF VARF GT 7 THEN VARF=.;
    IF VARG = 'less' OR VARG = 'LESS' OR VARG = 'Less' THEN VARH = 0;
    IF VARG = 'some' OR VARG = 'SOME' THEN VARH = 1;
    IF VARG = 'more' OR VARG = 'MORE' THEN VARH = 2;
RUN;
```

A: Example 3 is acceptable but it would be even better to complete all of these steps in a single data step. If it is intimidating to do too many things in one data step that's okay, you can split them up as long as you stay away from the overwriting demonstrated in the previous examples. You can also choose to run the data step a little at a time to be sure each line is working as expected. You can use comments to prevent some of the lines from executing and uncomment one at a time as you re-run the data step.

Tips:

- Never replace a permanent dataset unless absolutely necessary (`DATA NICOLE.ORIGINAL; SET NICOLE.ORIGINAL;`) – you should always keep the original file as it was received so you can retrace your steps if there is a problem.
- Even though it might *work*, try not to replace a dataset, even a temporary one, with another dataset of the same name (`DATA TEMP1; SET TEMP1;`) – it makes it quite difficult to debug when there is an error.
- Try using comments or just moving the RUN statement around to run a long data step in sections. This way, you can be more efficient with your code and still run the data step a little at a time to check for errors.

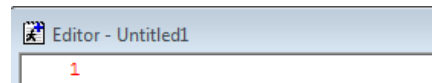
Naming your datasets

Tips:

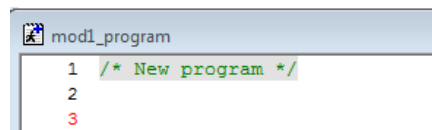
- Use names that are meaningful to you when creating datasets or variables (short letters and numbers seem easy in the moment but you will not remember what those represent if you come back to the program later.
- Names should be ≤ 32 characters in length (though there are some exceptions to this)
- Names must start with a letter or underscore (_) – they can never start with a number
- Names should only be made up of letters, numbers, or underscores
- Names are not case sensitive and can contain upper- and lowercase letters

Saving your work

When you first open SAS, it will create a new program for you (Editor – Untitled1). It is best to rename and save your program right away – this way if anything happens (accidental close, Citrix crash, etc.) you will not lose your work.



Let's say I type something and save this program as mod1_program.



Notice what happens when I start to type again. An asterisk appears next to the program name any time I have typed something new and the changes have not yet been saved. This symbol should remind you to routinely hit the save button throughout your SAS session.

